



University of Puerto Rico

Mayaguez campus

Department of Computer Science and Engineering

Undergraduate research in software (INSO 4998 - 001)

Fourth Report

Task 5: Applying measurements to circuits

Juan D. Guadalupe Rosado

Daniel C. Colon Rivera

Professor Venkataramani Kumar

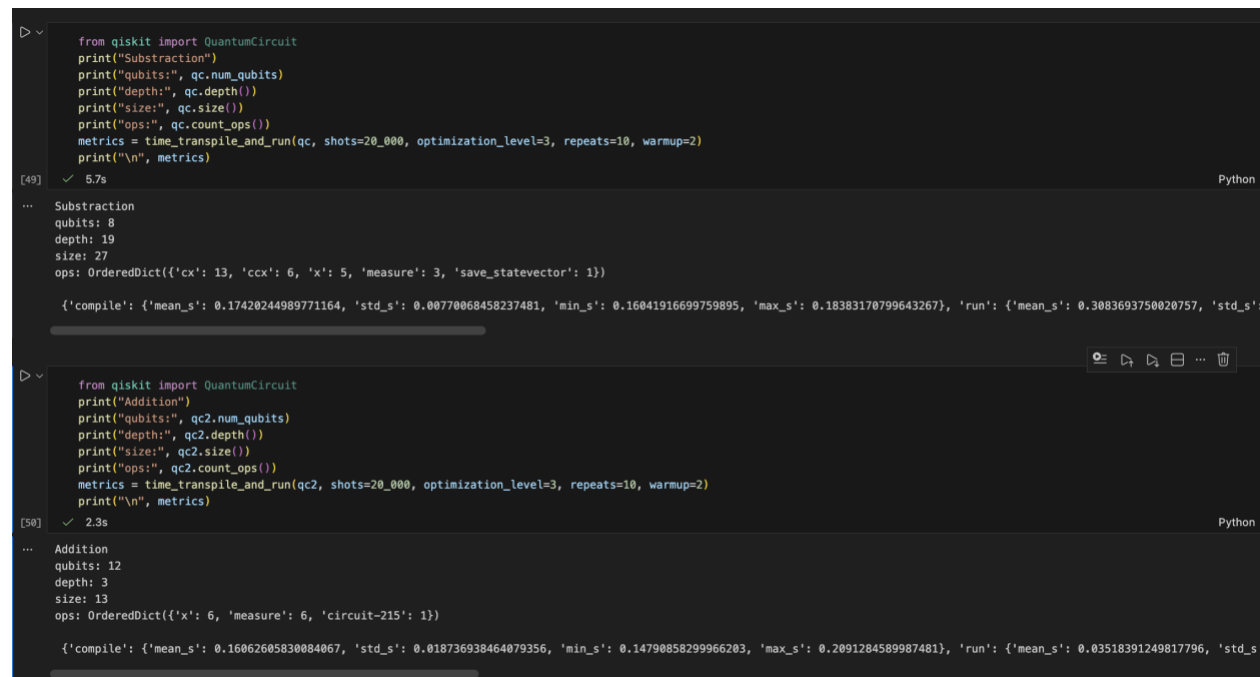
For this report we were tasked with measuring arithmetic quantum circuits applying any benchmarks we could find. For the benchmarks, I used the number of qubits in the circuit (Width), the number of sequential layers (Depth), the number of operations done in the circuit (size), like gates and measures, which would give us a tangible way of comparing circuits. For example, this is what my subtraction circuit returned once it was benchmarked:

```
qubits: 8
depth: 19
size: 27
ops: OrderedDict({'cx': 13, 'ccx': 6, 'x': 5, 'measure': 3, 'save_statevector': 1})
```

With a time of:

```
{'compile': {'mean_s': 0.17864879589978955, 'std_s': 0.0228589438397583, 'min_s': 0.1570153329957975, 'max_s': 0.2369187919975957}, 'run': {'mean_s': 0.26185839579984566, 'std_s': 0.06958227726111988, 'min_s': 0.18823349999566562, 'max_s': 0.34185839579984566}}
```

With this, we can compare and contrast different circuits we have on hand. This is a comparison between an arithmetic subtraction circuit and an arithmetic addition circuit:



The screenshot shows a Jupyter Notebook with two code cells. The first cell, labeled [49], defines a subtraction circuit and runs it. The output shows the circuit's properties and performance metrics. The second cell, labeled [50], defines an addition circuit and runs it, also showing its properties and performance metrics.

```
from qiskit import QuantumCircuit
print("Subtraction")
print("qubits:", qc.num_qubits)
print("depth:", qc.depth())
print("size:", qc.size())
print("ops:", qc.count_ops())
metrics = time_transpile_and_run(qc, shots=20_000, optimization_level=3, repeats=10, warmup=2)
print("\n", metrics)
```

Subtraction
qubits: 8
depth: 19
size: 27
ops: OrderedDict({'cx': 13, 'ccx': 6, 'x': 5, 'measure': 3, 'save_statevector': 1})

```
{'compile': {'mean_s': 0.17420244989771164, 'std_s': 0.00770068458237481, 'min_s': 0.16041916699759895, 'max_s': 0.18383170799643267}, 'run': {'mean_s': 0.3083693750020757, 'std_s': 0.06958227726111988, 'min_s': 0.18823349999566562, 'max_s': 0.34185839579984566}}
```

```
from qiskit import QuantumCircuit
print("Addition")
print("qubits:", qc2.num_qubits)
print("depth:", qc2.depth())
print("size:", qc2.size())
print("ops:", qc2.count_ops())
metrics = time_transpile_and_run(qc2, shots=20_000, optimization_level=3, repeats=10, warmup=2)
print("\n", metrics)
```

Addition
qubits: 12
depth: 3
size: 13
ops: OrderedDict({'x': 6, 'measure': 6, 'circuit-215': 1})

```
{'compile': {'mean_s': 0.16062605830084067, 'std_s': 0.018736938464079356, 'min_s': 0.14790858299966203, 'max_s': 0.2091284589987481}, 'run': {'mean_s': 0.03518391249817796, 'std_s': 0.00770068458237481, 'min_s': 0.02740858299966203, 'max_s': 0.04288391249817796}}
```

I'll try to get more circuits done in the future, as well as custom made circuits to see if we can start benchmarking for the poster/presentation.